Trabajo Práctico Final: Estrategia de Testing (Aplicación Clínica)

Versión 3.0

Integrantes: Ganduglia, Ciro. Rosal, Fernando Daniel. Sosa, Lucas. Stufano, Nazareno.

---

1. Presentación y Objetivos

El presente trabajo práctico tiene como propósito demostrar la adquisición de las capacidades fundamentales para el testing de un producto de software. El desarrollo exige estructurar las validaciones diferenciando estrictamente dos niveles arquitectónicos:

- **Testing atómico (unitario):** sobre componentes independientes en memoria sin dependencias externas.
- **Testing de integración:** sobre componentes acoplados con recursos externos (motor relacional de base de datos MySQL y concurrencia multihilo).

El diseño de los casos de prueba se realizó con técnicas de caja negra (particiones de equivalencia y análisis de valores límite) a partir de la especificación SRS. Las suites se implementaron con JUnit 4 y se ejecutaron con aserciones habilitadas (`-ea`). Según los criterios de entrega, una prueba en **verde** indica funcionamiento correcto y una prueba en **rojo** indica la detección de un error.

---

2. Consignas y Desarrollo

Consigna 1: Testing de la Clase AsociadoDTO

1. Escenarios

Nro de escenario	Descripción
1	AsociadoDTO recién creado con el constructor por defecto, sin datos cargados.
2	AsociadoDTO creado con el constructor con parámetros: ("Ana", "Gómez", "87654321", "Mar del Plata", "Belgrano", 456, "2239876543").

2. Tabla de Particiones

Datos de Prueba y Casos de Prueba (cada clase se identifica con un número entre paréntesis)

Condición de entrada	Clases Válidas	Clases Inválidas
nombre	(1) nombre != null	(2) nombre == null
apellido	(3) apellido != null	(4) apellido == null

Condición de entrada	Clases Válidas	Clases Inválidas
dni	(5) dni != null	(6) dni == null
ciudad	(7) ciudad != null	(8) ciudad == null
calle	(9) calle != null	(10) calle == null
numero	(11) numero ≥ 0	(12) numero < 0
telefono	(13) telefono != null	(14) telefono == null

3. Batería de Pruebas

Tipo de clase (correcta o incorrecta)	Valores de entrada	Salida esperada	Clases de prueba cubiertas
Correcta	Esc. 1: nombre = "Juan", apellido = "Pérez", dni = "12345678", ciudad = "Mar del Plata", calle = "San Martín", numero = 1234, telefono = "2231234567"	Cada getter devuelve el valor asignado	1, 3, 5, 7, 9, 11, 13
Correcta	Esc. 1: numero = 0 (valor límite)	getNumero() devuelve 0	11
Correcta	Esc. 2: se consultan todos los getters	Devuelven los valores pasados al constructor	1, 3, 5, 7, 9, 11, 13
Incorrecta	Esc. 1: nombre = null	AssertionError	2
Incorrecta	Esc. 1: apellido = null	AssertionError	4
Incorrecta	Esc. 1: dni = null	AssertionError	6
Incorrecta	Esc. 1: ciudad = null	AssertionError	8
Incorrecta	Esc. 1: calle = null	AssertionError	10
Incorrecta	Esc. 1: numero = -1 (valor límite)	AssertionError	12
Incorrecta	Esc. 1: telefono = null	AssertionError	14

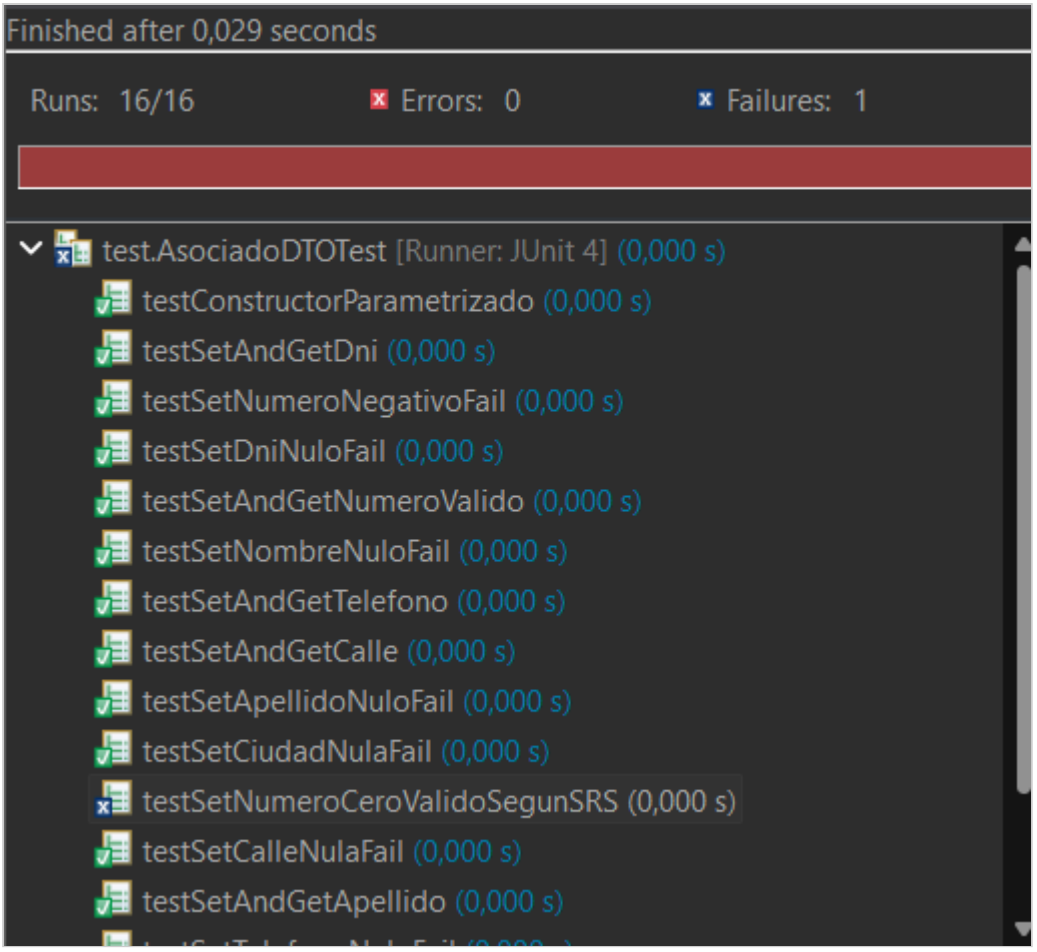
4. Aclaraciones

- Valores límite de numero: 0 (último valor válido según la SRS) y -1 (primer valor inválido).
- El caso numero = 0 detectó el defecto del código provisto: el setter exige numero > 0.
- Cada caso de setters parte del escenario 1 porque la suite crea un AsociadoDTO nuevo antes de cada prueba.
- Las validaciones están implementadas con assert, por lo que la suite se ejecuta con aserciones habilitadas (-ea).

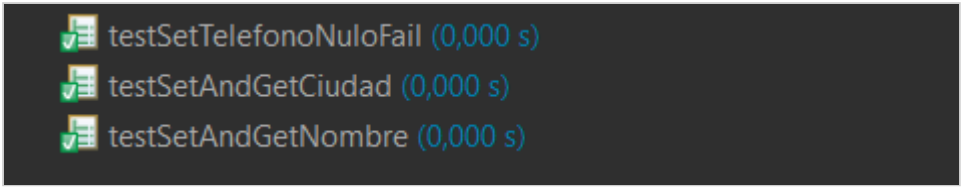
Apéndice A: Matriz de Casos de Prueba (AsociadoDTOTest)

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Propósito / Verificación
TC01	`testSetAndGetNombre`	nombre = "Juan"	Retorna "Juan"	Verificación de getter/setter de nombre.
TC02	`testSetAndGetApellido`	apellido = "Pérez"	Retorna "Pérez"	Verificación de getter/setter de apellido.
TC03	`testSetAndGetDni`	dni = "12345678"	Retorna "12345678"	Verificación de getter/setter de dni.
TC04	`testSetAndGetCiudad`	ciudad = "Mar del Plata"	Retorna "Mar del Plata"	Verificación de getter/setter de ciudad.
TC05	`testSetAndGetCalle`	calle = "San Martín"	Retorna "San Martín"	Verificación de getter/setter de calle.
TC06	`testSetAndGetNumeroValido`	numero = 1234	Retorna 1234	Verificación de asignación de número positivo.
TC07	`testSetAndGetTelefono`	telefono = "2231234567"	Retorna "2231234567"	Verificación de getter/setter de telefono.
TC08	`testConstructorParametrizado`	Todos los campos válidos	Estado interno coincidente	Verificación del constructor con parámetros.
TC09	`testSetNombreNuloFail`	nombre = null	Lanza AssertionError	Verificación de control contra valores nulos.
TC10	`testSetApellidoNuloFail`	apellido = null	Lanza AssertionError	Verificación de control contra valores nulos.
TC11	`testSetDniNuloFail`	dni = null	Lanza AssertionError	Verificación de control contra valores nulos.
TC12	`testSetNumeroNegativoFail`	numero = -1	Lanza AssertionError	Rechazo de valores enteros negativos (valor límite inferior de la clase inválida).
TC13	`testSetNumeroCeroValidoSegunSRS`	numero = 0	Asignación exitosa (0)	Validación del valor límite 0 según especificación SRS (SRS 3.1).
TC14	`testSetCiudadNulaFail`	ciudad = null	Lanza AssertionError	Verificación de control contra valores nulos.
TC15	`testSetCalleNulaFail`	calle = null	Lanza AssertionError	Verificación de control contra valores nulos.
TC16	`testSetTelefonoNuloFail`	telefono = null	Lanza AssertionError	Verificación de control contra valores nulos.

Resultados de ejecución (código provisto)



Inciso 1.1 — Ejecución de AsociadoDTOTest sobre el código provisto: 16 pruebas, 1 falla (testSetNumeroCeroValidoSegunSRS).



Inciso 1.2 — Continuación de la ejecución anterior (pruebas restantes).

Conclusiones Consigna 1

1. **Aislamiento del Test:** La suite AsociadoDTOTest evalúa la entidad AsociadoDTO en memoria sin requerir la instanciación de servicios externos, conexiones a MySQL ni capas de interfaz gráfica.
2. **Sensibilidad a la Configuración de la JVM:** La validación mediante la sentencia `assert` requiere explícitamente la bandera `-ea` en el entorno de ejecución para actuar de forma efectiva. En entornos productivos donde las aserciones suelen desactivarse, es recomendable sustituir las aserciones nativas por excepciones explícitas (`IllegalArgumentException`).
3. **Consistencia con los Requisitos (SRS):** Se detectó que el setter de numero evalúa `numero > 0`, mientras que la especificación SRS establece que el valor debe ser mayor o igual a cero (`numero >= 0`). La prueba de valor límite TC13 (numero = 0) expuso la inconsistencia. La corrección se aplicó al inicio de la Consigna 2, según lo requerido por el enunciado (ver captura Inciso 1.3).

Código Fuente

El código fuente completo de la suite se encuentra en el repositorio del grupo (`https://github.com/Nazastufano/Grupo10-Testing-Clinica`):

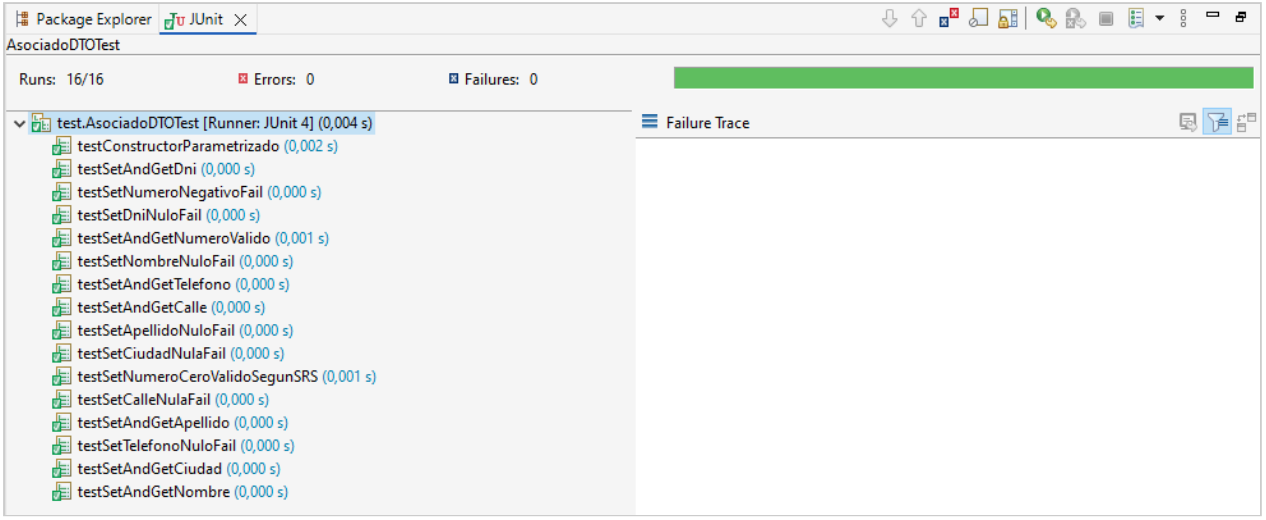
- `Info/TPE2026/src/main/java/test/AsociadoDTOTest.java`

Se incluye además una copia de los archivos en la carpeta `Clases Test/java/` del repositorio, para su lectura directa desde GitHub.

Consigna 2: Testing de Integración (AsociadoDTO, DataAccessObject y BaseDeDatos)

Corrección del Código del Inciso 1

Según lo requerido por la consigna, antes de ejecutar las pruebas de integración se corrigió el defecto detectado en la Consigna 1: el setter `setNumero` de `AsociadoDTO` evaluaba `assert numero > 0;`, mientras que la SRS establece que el número debe ser mayor o igual a cero. Se modificó la condición a `assert numero >= 0;` y se re-ejecutó la suite AsociadoDTOTest, obteniéndose 16/16 pruebas exitosas.



Inciso 1.3 — Re-ejecución de AsociadoDTOTest tras la corrección: 16/16 pruebas exitosas.

1. Análisis de Código y Detección de Inconsistencias

- **Gestión de Conexión en BaseDeDatos:** La URL configurada es `jdbc:mysql://localhost:3306/?useSSL=false&serverTimezone=UTC`. El método `iniciarBD()` crea la base de datos Grupo_7 si no existe, ejecuta `USE Grupo_7;`, elimina la tabla Asociados (`DROP TABLE IF EXISTS`) y la recrea poblándola con 20 datos semilla.
- **Mapeo y Consultas SQL en DataAccessObject:** La consulta INSERT utiliza DNI en mayúsculas (`INSERT INTO Asociados(nombre, apellido, DNI, calle, numero, ciudad, telefono)`), mientras que eliminar utiliza `WHERE dni = ?`. El método CrearDTO mapea los campos desde el ResultSet.
- **Inconsistencias y Reglas de Integridad:** La columna DNI posee restricción UNIQUE en MySQL. Insertar un DNI duplicado dispara `java.sql.SQLIntegrityConstraintViolationException`.
- **Lectura por ID en DataAccessObject:** El método `cargar(int id)` accede a las columnas del ResultSet sin invocar previamente `resultado.next()`, por lo que el cursor permanece antes de la

primera fila y se lanza `java.sql.SQLException` ("Before start of result set").

- **Actualización en BaseDeDatos:** El método `ejecutarActualizacion` asigna el parámetro de la cláusula `WHERE id = ?` con un valor fijo (`stmt.setInt(8, 1)`), por lo que toda actualización se aplica sobre el registro con id 1, independientemente del asociado indicado. Al asignarle a ese registro el DNI de otro asociado existente, se viola la restricción UNIQUE de la columna DNI.
- **Manejo de errores de conexión:** `abrirConexion()` captura la `SQLException` sin propagarla, dejando la conexión en `null`. Ante credenciales inválidas, el error se manifiesta luego como `NullPointerException` en `iniciarBD()`, ocultando la causa original.

2. Escenarios

Nro de escenario	Descripción
1	MySQL activo en localhost:3306 y base Grupo_7 inicializada con iniciarBD(): tabla Asociados con 20 asociados semilla (id 1: Juan, DNI "12345678"; Maria Template, DNI "87654321").
2	MySQL activo sin haber ejecutado iniciarBD(): la base Grupo_7 o la tabla Asociados no existen.

3. Tabla de Particiones

Datos de Prueba y Casos de Prueba (cada clase se identifica con un número entre paréntesis)

Condición de entrada	Clases Válidas	Clases Inválidas
Instancia de BaseDeDatos (getInstancia)	(1) Devuelve siempre la misma instancia	(2) Instancias distintas o referencia nula
Conexión (getConexion)	(3) Servidor activo y credenciales correctas	(4) Credenciales incorrectas o servidor detenido
Inicialización (iniciarBD)	(5) Se crean el esquema y la tabla con los 20 registros semilla	—
Asociado a guardar (guardar)	(6) Asociado válido con DNI no registrado	(7) null (8) DNI ya registrado
limite (cargarConLimite)	(9) limite > 0	(10) limite ≤ 0
id (cargar)	(11) id > 0 de un asociado existente	(12) id ≤ 0 (13) id > 0 inexistente
dni (eliminar)	(14) dni de un asociado existente	(15) null
Asociado a actualizar (actualizar)	(16) Asociado existente (DNI registrado) con datos válidos	(17) null

4. Batería de Pruebas

Tipo de clase (correcta o incorrecta)	Valores de entrada	Salida esperada	Clases de prueba cubiertas
Correcta	Esc. 1: getInstancia() invocado dos veces	Devuelve la misma referencia, no nula	1

Tipo de clase (correcta o incorrecta)	Valores de entrada	Salida esperada	Clases de prueba cubiertas
Correcta	Esc. 1: getConexion()	Conexión no nula y abierta	3
Correcta	Esc. 2: iniciarBD()	cargarBD() devuelve 20 asociados; el primero es Juan, DNI "12345678"	5
Correcta	Esc. 1: guardar(Roberto Gomez, DNI "99887766")	cargarBD() devuelve 21 asociados	6
Correcta	Esc. 1: cargarConLimite(5)	Devuelve 5 asociados	9
Correcta	Esc. 1: cargar(1)	Devuelve a Juan, DNI "12345678"	11
Correcta	Esc. 1: actualizar(Maria Template, DNI "87654321", teléfono "2239990000")	Se actualiza el teléfono del asociado DNI "87654321"; el asociado DNI "12345678" no cambia	16
Incorrecta	Esc. 1: eliminar("12345678") y luego cargar(1)	cargarBD() devuelve 19 asociados y cargar(1) devuelve null	14, 13
Incorrecta	Esc. 1: guardar(null)	AssertionError	7
Incorrecta	Esc. 1: guardar(Pedro Duplicado, DNI "12345678")	SQLException (DNI duplicado)	8
Incorrecta	Esc. 1: cargarConLimite(0) (valor límite)	AssertionError	10
Incorrecta	Esc. 1: cargar(-1)	AssertionError	12
Incorrecta	Esc. 1: eliminar(null)	AssertionError	15

5. Aclaraciones

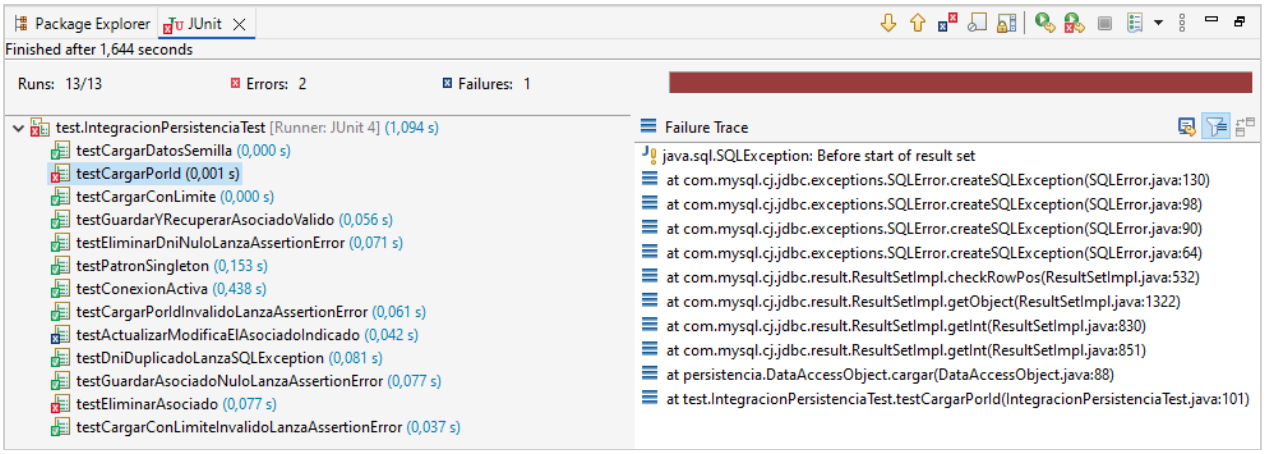
- Todos los casos salvo el de iniciarBD() parten del escenario 1, porque la suite ejecuta iniciarBD() antes de cada prueba.
- Valores límite: limite = 0 (primer valor inválido) e id = -1.
- Las clases (2), (4) y (17) no se cubren: el Singleton no permite crear otra instancia desde fuera de la clase, probar credenciales incorrectas exige modificar las fijadas en BaseDeDatos o detener el servidor, y actualizar(null) no se incluyó en la suite.
- Los casos de cargar(1), eliminar y actualizar detectaron defectos del código provisto (ver Apéndice B).

Apéndice B: Matriz de Resultados de Pruebas de Integración (IntegracionPersistenciaTest)

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
IT01	`testPatronSingleton`	BaseDeDatos.getInstancia()	Instancia única compartida	Identidad de referencia constatada con assertSame.	**PASS (Verde)**
IT02	`testConexionActiva`	bd.getConexion()	Conexión JDBC abierta	Conexión establecida a MySQL en localhost:3306.	**PASS (Verde)**

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
IT03	`testCargarDatosSemilla`	Ejecución de iniciarBD()	20 registros iniciales	Esquema y 20 registros semilla creados correctamente.	**PASS (Verde)**
IT04	`testGuardarYRecuperarAsociadoValido`	Asociado con DNI "99887766"	Persistencia exitosa en BD	Registro insertado vía PreparedStatement y verificado en cargarBD().	**PASS (Verde)**
IT05	`testCargarConLimite`	limite = 5	Retorno de 5 DTOs	SELECT * FROM Asociados LIMIT 5 mapeado correctamente.	**PASS (Verde)**
IT06	`testCargarPorId`	id = 1	AsociadoDTO id = 1	ERROR: SQLException "Before start of result set" en DataAccessObject.cargar (línea 88): se lee el ResultSet sin invocar next().	**ERROR (Rojo)**
IT07	`testEliminarAsociado`	dni = "12345678"	Registro eliminado en BD	La eliminación impacta correctamente (cargarBD() retorna 19 registros). El error se produce en la verificación posterior con cargar(1): SQLException "Illegal operation on empty result set", mismo defecto que IT06.	**ERROR (Rojo)**
IT08	`testDniDuplicadoLanzaSQLException`	Reuso de DNI "12345678"	Captura de SQLException	Violación de restricción UNIQUE en MySQL atrapada.	**PASS (Verde)**
IT09	`testGuardarAsociadoNuloLanzaAssertionError`	Asociado = null	Lanza AssertionError	Control defensivo de objeto nulo validado.	**PASS (Verde)**
IT10	`testCargarConLimiteInvalidoLanzaAssertionError`	limite = 0	Lanza AssertionError	Precondición limite > 0 validada.	**PASS (Verde)**
IT11	`testCargarPorIdInvalidoLanzaAssertionError`	id = -1	Lanza AssertionError	Precondición id > 0 validada.	**PASS (Verde)**
IT12	`testEliminarDniNuloLanzaAssertionError`	dni = null	Lanza AssertionError	Precondición dni != null validada.	**PASS (Verde)**
IT13	`testActualizarModificaElAsociadoIndicado`	Asociado existente DNI "87654321" con teléfono nuevo	Se actualiza el teléfono del asociado indicado y no se modifica ningún otro registro	FALLO DETECTADO: actualizar() lanza SQLException "Duplicate entry '87654321' for key DNI": la sentencia se aplica sobre el registro id = 1 (stmt.setInt(8, 1)) en lugar del asociado indicado.	**FAIL (Rojo)**

> JUnit distingue **Error** (el código bajo prueba lanza una excepción no esperada por el test, IT06 e IT07) de **Failure** (una aserción del test no se cumple, IT13). Ambos casos corresponden a defectos detectados por la suite.



Inciso 2 — Ejecución de IntegracionPersistenciaTest contra MySQL: 13 pruebas, 2 errores y 1 falla (defectos detectados en cargar() y actualizar()).

Conclusiones Consigna 2

1. **Validación de la Infraestructura de Persistencia:** Se confirmó el correcto funcionamiento del patrón Singleton en BaseDeDatos, la conectividad con el motor MySQL local, la creación automática de tablas e inyección de datos semilla, y la protección contra duplicados mediante la restricción de unicidad del DNI.
2. **Detección de Fallos en el Código Provisto:** La suite atrapó dos defectos en las operaciones CRUD: la lectura individual por clave primaria (`cargar` no invoca `next()` sobre el ResultSet), que hace fallar a testCargarPorId y a la verificación final de testEliminarAsociado —la eliminación en sí funciona correctamente—, y la actualización (`actualizar` modifica siempre el registro id = 1), detectada por testActualizarModificaElAsociadoIndicado. Esto demuestra el valor del testing de integración para detectar inconsistencias antes de la puesta en producción.
3. **Corrección del Inciso 1:** La corrección de la condición de `setNumero` (`numero >= 0`) fue validada re-ejecutando AsociadoDTOTest con resultado 16/16 exitoso.

Código Fuente

El código fuente completo de la suite se encuentra en el repositorio del grupo (`https://github.com/Nazastufano/Grupo10-Testing-Clinica`):

- `Info/TPE2026/src/main/java/test/IntegracionPersistenciaTest.java`

Se incluye además una copia de los archivos en la carpeta `Clases Test/java/` del repositorio, para su lectura directa desde GitHub.

Consigna 3: Testing de Integración con Mocks y Concurrencia

1. Preparación: Errores de Compilación Detectados

Al ejecutar la suite se detectaron dos errores de compilación en el código provisto, que impedían instanciar los estados afectados (JUnit reportaba `java.lang.Error: Unresolved compilation problem`). Se corrigieron para poder ejecutar las pruebas:

- El archivo `TrasladandoPacienteClinicasState.java` no coincidía con el nombre de la clase pública que declara (`TrasladandoPacienteClinicaState`). Se renombró el archivo.

- `EnTallerState.java` declaraba el paquete `model.modeloDominio.ambulancia` en lugar de `modelo.modeloDominio.ambulancia`. Se corrigió la declaración.

2. Escenarios

Nro de escenario	Descripción
1	Ambulancia recién creada: estado DisponibleState, no ocupada y simulación activa.
2	Ambulancia con un doble de prueba (StateMock) inyectado como estado actual.
3	Ambulancia ocupada, con un hilo independiente solicitando servicio.
4	Ambulancia en estado TrasladandoPacienteClinicaState.

3. Tabla de Particiones

Datos de Prueba y Casos de Prueba (cada clase se identifica con un número entre paréntesis)

Condición de entrada	Clases Válidas	Clases Inválidas
Delegación al State (atenderDomicilio, trasladarALaClinica, regresarSinPaciente)	(1) El contexto invoca el método correspondiente del State	(2) No se invoca el método del State o se invoca otro
Hilo RetornoAutomatico	(3) Con la simulación activa, el hilo permanece en ejecución	(4) El hilo finaliza con la simulación activa
Retorno desde TrasladandoPacienteClinicaState	(5) Transición a DisponibleState	(6) Transición a cualquier otro estado
Acceso concurrente con la ambulancia ocupada	(7) El hilo solicitante espera (wait) y se libera con notifyAll()	(8) El hilo no espera o queda bloqueado indefinidamente
Flujo completo de la máquina de estados (domicilio y taller)	(9) Secuencia de estados conforme al diseño y flag ocupado coherente	(10) Estado o flag ocupado inconsistente en algún paso

4. Batería de Pruebas

Tipo de clase (correcta o incorrecta)	Valores de entrada	Salida esperada	Clases de prueba cubiertas
Correcta	Esc. 2: atenderDomicilio(asociado)	Se invoca SolicitudDeAtencionDomicilio() del State; ocupado = true	1
Correcta	Esc. 2: trasladarALaClinica(asociado)	Se invoca SolicitudDeTraslado() del State; ocupado = true	1
Correcta	Esc. 2 con ocupado = true: regresarSinPaciente()	Se invoca RetornoClinica() del State; ocupado = false	1

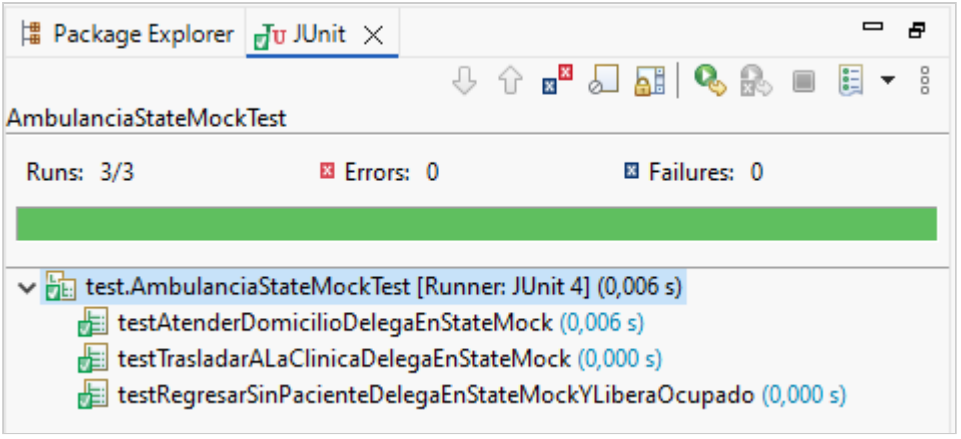
Tipo de clase (correcta o incorrecta)	Valores de entrada	Salida esperada	Clases de prueba cubiertas
Correcta	Esc. 1: se inicia el hilo RetornoAutomatico	El hilo sigue en ejecución mientras la simulación está activa	3
Correcta	Esc. 4: regresarSinPaciente()	La ambulancia queda en DisponibleState	5
Correcta	Esc. 3: un hilo secundario invoca atenderDomicilio(asociado)	El hilo queda en espera; tras regresarSinPaciente() se libera y finaliza	7
Correcta	Esc. 1: atenderDomicilio → regresarSinPaciente → regresarSinPaciente → solicitarMantenimiento → volviendoDelTaller → regresarSinPaciente	Disponible → AtendiendoEnDomicilio → RegresandoClinicaSinPaciente → Disponible → EnTaller → RegresandoDelTaller → Disponible, con ocupado coherente en cada paso	9

5. Aclaraciones

- Las clases inválidas (2), (4), (6), (8) y (10) describen comportamientos defectuosos: cada caso verifica la clase válida y falla (rojo) si el sistema cae en la inválida. Así se detectaron (4) y (6).
- StateMock registra qué método del State se invocó, sin depender de las implementaciones concretas de los estados.
- Los métodos probados no recorren colecciones: los escenarios se definen por el estado de la ambulancia (estado actual, ocupación, simulación activa e hilos en espera).

Apéndice C1: Matriz de Pruebas Atómicas con Mocks (AmbulanciaStateMockTest)

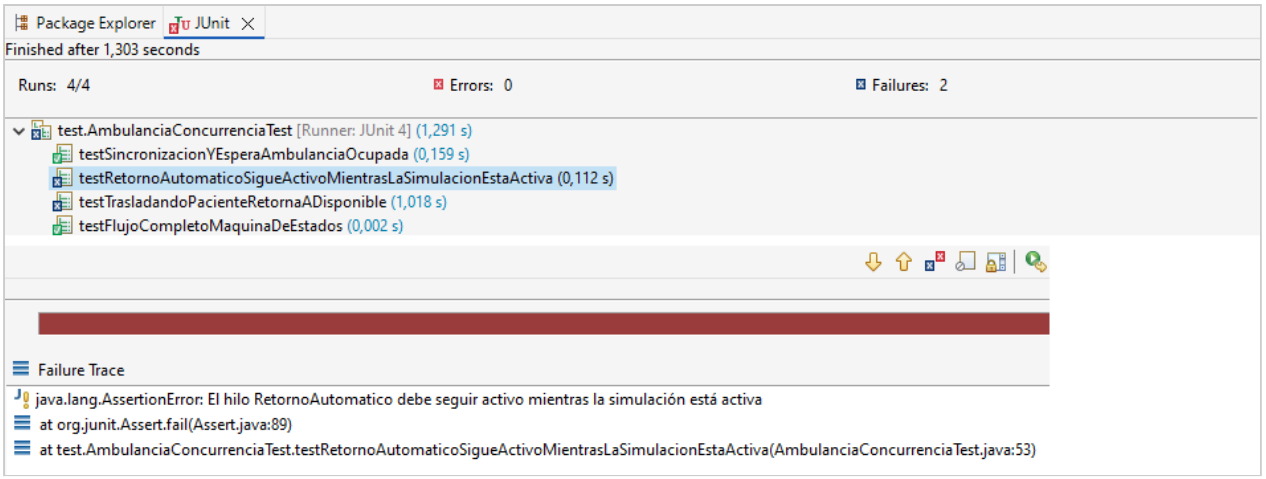
ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
TC-M01	`testAtenderDomicilioDelegaEnStateMock`	Inyección de StateMock	Delegación del método en el State	Delegación hacia SolicitudDeAtencionDomicilio() confirmada.	**PASS (Verde)**
TC-M02	`testTrasladarALaClinicaDelegaEnStateMock`	Inyección de StateMock	Delegación del método en el State	Delegación hacia SolicitudDeTraslado() confirmada.	**PASS (Verde)**
TC-M03	`testRegresarSinPacienteDelegaEnStateMockYLiberaOcupado`	ocupado = true, Mock inyectado	Libera flag ocupado y notifica	Ejecuta RetornoClinica() y asigna isOcupado() = false.	**PASS (Verde)**



Inciso 3.1 — Ejecución de AmbulanciaStateMockTest: 3/3 pruebas exitosas.

Apéndice C2: Matriz de Integración y Concurrencia (AmbulanciaConcurrenciaTest)

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
TC-C01	`testRetornoAutomaticoSigueActivoMientrasLaSimulacionEstaActiva`	isSimulacionActiva() = true; se inicia el hilo RetornoAutomatico	El hilo permanece en ejecución mientras la simulación está activa (clase (3))	FALLO DETECTADO: el hilo finaliza de inmediato por la condición invertida while (!ambulancia.isSimulacionActiva()) en RetornoAutomatico.	**FAIL (Rojo)**
TC-C02	`testSincronizacionYEsperaAmbulanciaOcupada`	ocupado = true, hilo secundario pidiendo servicio	Bloqueo en wait() y posterior liberación con notifyAll() (clase (7))	Sincronización exitosa y gestión segura de concurrencia.	**PASS (Verde)**
TC-C03	`testTrasladandoPacienteRetornaADisponible`	Estado TrasladandoPacienteClinicaState	Transición a DisponibleState (clase (5))	FALLO DETECTADO: la ambulancia conmuta erróneamente a RegresandoDelTallerState.	**FAIL (Rojo)**
TC-C04	`testFlujoCompletoMaquinaDeEstados`	Ambulancia recién creada (Disponible); Asociado y Operario válidos	Disponible → AtendiendoEnDomicilio → RegresandoClinicaSinPaciente → Disponible → EnTaller → RegresandoDelTaller → Disponible, con ocupado coherente en cada paso (clase (9))	Secuencia de estados y flag ocupado conformes al diseño en ambos ciclos.	**PASS (Verde)**



Inciso 3.2 — Ejecución de AmbulanciaConcurrenciaTest: 4 pruebas, 2 fallas (defectos de concurrencia y de estado detectados).

Conclusiones Consigna 3

1. **Aislamiento mediante Mocks:** La construcción del doble de prueba StateMock permitió validar el acoplamiento débil del patrón State con la clase Ambulancia. Se verificó que los métodos del contexto delegan correctamente el comportamiento sobre la interfaz sin depender de las implementaciones concretas.
2. **Evaluación de Concurrencia, Sincronización y Flujo Completo:** Se constató la efectividad de los bloques `synchronized` y las llamadas a `wait()` / `notifyAll()` en la clase Ambulancia, y se validó el flujo completo de la máquina de estados en los ciclos de atención a domicilio y de mantenimiento.
3. **Descubrimiento de Fallos de Estado y Concurrencia:** Las pruebas cumplieron con el objetivo de validación al atrapar dos defectos: la finalización prematura del hilo RetornoAutomatico debido a la condición invertida de su bucle (`while (!ambulancia.isSimulacionActiva())`), y la transición errónea en TrasladandoPacienteClinicaState, que desvía la ambulancia a RegresandoDelTallerState en lugar de dejarla disponible.
4. **Errores de Compilación:** Se detectaron y corrigieron el nombre de archivo de TrasladandoPacienteClinicaState y el paquete de EnTallerState, que impedían ejecutar la suite.
5. **Criterio de las Aserciones:** Las pruebas verifican el comportamiento correcto esperado según el diseño; por lo tanto, resultan en rojo mientras los defectos existen y en verde ante una implementación correcta.

Código Fuente

El código fuente completo de la suite se encuentra en el repositorio del grupo (`https://github.com/Nazastufano/Grupo10-Testing-Clinica`):

- `Info/TPE2026/src/main/java/test/AmbulanciaStateMockTest.java` (incluye el doble de prueba StateMock)
- `Info/TPE2026/src/main/java/test/AmbulanciaConcurrenciaTest.java`

Se incluye además una copia de los archivos en la carpeta `Clases Test/java/` del repositorio, para su lectura directa desde GitHub.

3. Conclusiones Generales del Informe

1. **Cumplimiento Integral de los Criterios de Entrega:** Se entregó el código fuente completo de las cuatro suites de pruebas (AsociadoDTOTest, IntegracionPersistenciaTest, AmbulanciaStateMockTest y AmbulanciaConcurrenciaTest) en el repositorio del grupo, y se documentaron en este informe las matrices formales de escenarios y particiones de equivalencia, las matrices de resultados observados en JUnit y las capturas de ejecución en verde (funcionamiento correcto) y en rojo (detección de errores).
2. **Detección Efectiva de Inconsistencias y Bugs:** Las suites diseñadas demostraron la capacidad del testing estructurado para atrapar las inconsistencias presentes en los tres incisos del TP:
    - Inconsistencia de valor límite entre el setter de AsociadoDTO y la especificación SRS (corregida y validada).
    - Falla en la lectura por ID (`cargar` sin `next()`) y en la actualización (`actualizar` sobre id fijo) en la integración con MySQL.
    - Bucle invertido en el hilo concurrente RetornoAutomatico y transición errónea en la máquina de estados.
    - Errores de compilación en TrasladandoPacienteClinicaState y EnTallerState.
3. **Validación Arquitectónica:** Se confirmó la robustez de los patrones de diseño aplicados en el sistema: Singleton en la conexión, DAO/DTO para desacoplamiento de persistencia, State mediante dobles de prueba (Mocks), y exclusión mutua concurrente con `synchronized`, `wait()` y `notifyAll()`.